

GroupA01_ProjectReport.docx

 Shri Vile Parle Kelavani Mandal

Document Details

Submission ID

trn:oid:::9832:151429668

Submission Date

Sep 11, 2026, 7:28 PM GMT+5:30

Download Date

Sep 11, 2026, 7:31 PM GMT+5:30

File Name

GroupA01_ProjectReport.docx

File Size

937.6 KB

10 Pages

3,431 Words

18,199 Characters

0% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **1 Not Cited or Quoted 0%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 
1 Not Cited or Quoted 0%
 Matches with neither in-text citation nor quotation marks
- 
0 Missing Quotations 0%
 Matches that are still very similar to source material
- 
0 Missing Citation 0%
 Matches that have quotation marks, but no in-text citation
- 
0 Cited and Quoted 0%
 Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 0%  Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1

Submitted works

Tilburg University on 2026-05-25

<1%



SCHOOL OF
BUSINESS MANAGEMENT



Big Data Analytics

Topic- Rating-Aware Chess Analytics



Submitted To:
Abhishek Sarkar
School of Business Management, SVKM's NMIMS,
Mumbai

Submitted By: BA – Group A01

Arjun Bhatnagar – A044 	Manav Karia – A061 
Akshat Rathod – A050 	Sarvesha Bhatgare – A078 
Aditi Chandra Yadav – A052 	Jasreen Kour Bagga – A086 

EXECUTIVE SUMMARY

Nowadays a chess coaching platform will provide two things: the objectively best move, but not the explanation of what it means to a player of a given level; and generic instruction content that will be the same for any user. This project aimed to determine if historical game data could fill this gap between the two.

This example demonstrated a full analytics pipeline from ingestion of raw data from Lichess, through data cleaning, feature engineering, two machine learning approaches and an interactive demonstration of the results to answer three executive questions:

- Does a rating advantage apply to all skill levels?
- Do there exist separate categories of games that require different coaching?
- Can one predict game outcomes based on available metadata?

The take-away message is clear: ratings make a difference. The winning percentage of the favorite player increases from 65.8% in the under-1200 group to 78.8% at 2400+ and this impact grows even greater at certain positions; a more detailed analysis of 765,433 engine evaluated games reveals that it intensifies at certain key positions, but nearly vanishes in the balanced ones. The classification modeling demonstrated that rating differential alone contained nearly all the predictive information in outcomes, while clustering revealed that 29% of the games were lost due to time management and not skill at chess. These results help build a strong business case: you can provide guidance that is much more relevant if you differentiate your coaching advice by rating band and failure pattern than if you take a one size fits all approach.

BUSINESS CONTEXT

Imagine a medium sized online chess school that provides coaching in a similar fashion to larger schools who make the engine analysis available free of charge. The challenge that it faces is that of differentiation – why should a player pay for coaching insight when a free engine will tell them the best move in any position?

This project suggests that the answer is that raw engine output is "rating-agnostic": that is, the same evaluation number represents something different to different players. A rating-aware layer that is based less on the engine's theory and more on the past results of players with a similar rating can tell a player not only what the best move is, but how often a player of their rating level is able to turn this advantage into a win. It is this distinction that forms the bedrock of a truly differentiated coaching product, and it's this project that aimed to create and validate.

DATA UNDERSTANDING – THE 5Vs

Dimension	Description
Volume	6,136,419 games in the base dataset and 765,433 games with move-level engine evaluations in the extended analysis.
Velocity	Monthly batch archive rather than a live stream, chosen deliberately for reproducibility over real-time ingestion.
Variety	Every rating band, every standard time control, and hundreds of distinct opening lines represented.
Veracity	Approximately 1% of games removed during cleaning (aborted games and unrealistic move counts) and the remainder validated against known chess statistics.
Value	Direct, monetizable coaching insight, rating-aware win probability, playing-style segmentation, and opening-level risk flags.

ENTERPRISE ARCHITECTURE

The data comes from the Lichess open database, is preprocessed locally, is then processed by the large-scale data processing layer Databricks, and then is returned to a lightweight local demo layer:

- The Lichess open database is the raw monthly PGN archive which is compressed and downloaded only once to be reproduced.
- Local preprocessing: The pre-processing of the raw archive into a structured table (using a custom regular-expression based parser which does not require full chess-move validation!)
- Databricks / DBFS is the cloud workspace and storage for all large-scale processing.
- At full data scale, cleaning, feature engineering, and aggregate analysis will be done using PySpark & Spark SQL.
- Spark MLlib → classification and clustering models
- Stockfish-powered live chessboard, which is only connected back to Databricks via exported calibration tables, local demo layer

It is important to note that this project does not run HDFS and Sqoop directly. HDFS is described architecturally as the production-scale equivalent of DBFS — the storage layer a live, continuously ingesting deployment would use, while DBFS serves that same role at this project's scale. Sqoop is used to move data from a relational source into Hadoop; this project's data came in as a static bulk export and is not from a live relational source, so it does not apply to Sqoop. Both are considered explicitly and not left unmentioned, to show that they are known to the architect regardless of what was executed.

DATA ENGINEERING

PARSING

The raw archive was read with a simple script that does not rely on a complete chess-move-validation library, but rather with a lightweight regex-based parser. The move-legality check was not performed in a first attempt but was traded for speed as an initial attempt using full validation was projected to take days to complete on the full archive; in this phase only game-level metadata was needed, not move sequences.

CLEANING

Two problems with the data were found and fixed: 60,000 or so games had unrealistic numbers of moves (most of these were above 200 and would almost certainly be small numbers of parsing artifacts rather than games), and 1457 games were recorded as having no result (aborted games). 6,075,566 games were remaining after cleaning, representing about 1% of the total.

FEATURE ENGINEERING

Feature	Construction	Purpose
rating_diff	White rating – Black rating	Signed measure of rating advantage
rating_band	Binned into 5 brackets (<1200 to 2400+)	Business-interpretable skill segment
outcome	Simplified from raw result notation	Clean classification target
base_time / increment	Split from the combined time_control field	Usable numeric time-control features
One-hot encodings	Applied to rating_band, opening, time_control	Converts categorical data for MLlib
eval_bucket	Binned engine evaluation into 7 ranges	Basis for the calibration analysis

MACHINE LEARNING

FINDING 1 -THE WIN CONVERSION CURVE

Filtering to games with a rating gap over 100 points and measuring how often the favored player actually won, by rating band:

Rating Band	Favored-Player Win Rate
<1200	65.8%
1200–1600	68.9%
1600–2000	71.8%
2000–2400	74.8%
2400+	78.8%

This is the project's core empirical evidence that a rating advantage does not mean the same thing at every skill level, it becomes measurably more reliable to convert as rating increases.

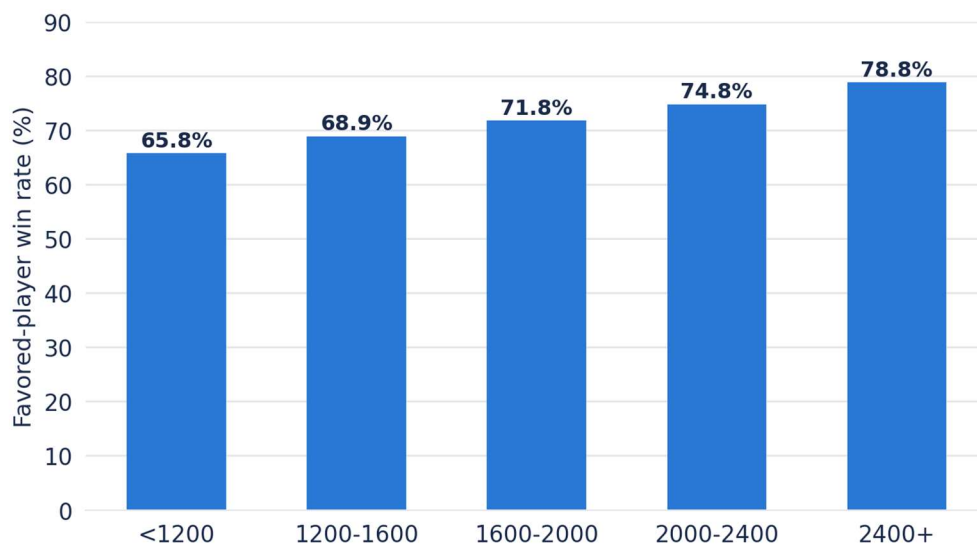


Figure 1 - Favored-player win rate by rating band

FINDING 2 -THE CLASSIFICATION

Three approaches were compared for predicting game outcome from rating differential, opening, and time control, using an identical train/test split:

Approach	Accuracy
Naive baseline (favored player always wins)	63.20%
Logistic regression	63.39%
Random forest	63.01%

The random forest model actually underperformed both alternatives, and this was not an experiment gone wrong but rather an informative result. The feature space was very sparse (about 3,766 dimensions, due to the many different opening sets being represented by one-hot encoding), and tree-based models are well-known to have trouble finding useful splits in sparse, high dimensional data, whereas linear models such as logistic regression work better with high

dimensional data. The overall finding is that the rating differential contains almost all of the predictive information in this set of features; the rest is noise.

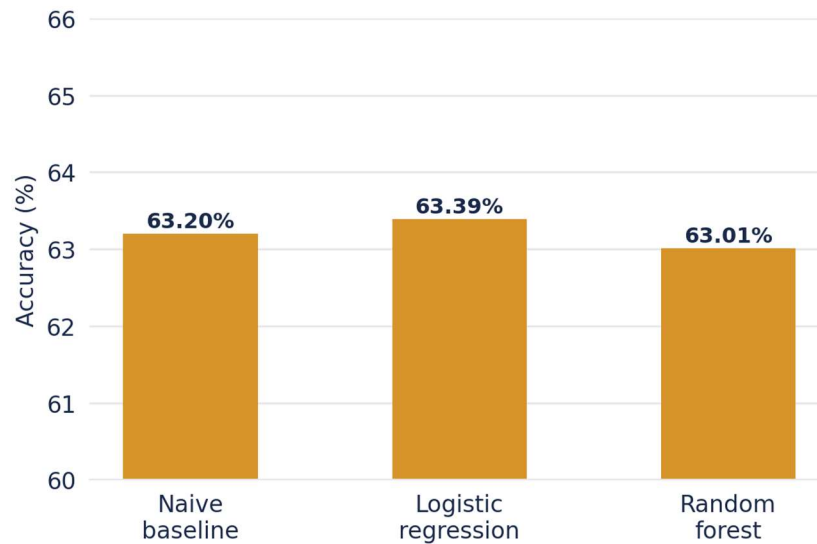


Figure 2 - Classification accuracy: baseline vs. logistic regression vs. random forest

FINDING 3 – CLUSTERING

Classification asks, who will win? clustering asks a different question entirely, without telling the model what to look for, do 6 million games naturally have categories which can be recognized, and are they distinct?

Five features describing how a game proceeded, but not who won it were used in kMeans clustering (k=4): stronger player's rating, total number of moves played in the game, base time and increment of the time control, and final result (checkmate, resignation, time forfeit, or abandonment encoded as numbers). All features were scaled by StandardScaler before clustering as KMeans is measuring raw distance between data points, it is important to perform this step because a feature with a large absolute range, such as move count (0–200), would have completely overwhelmed a feature with a small absolute range, such as increment (0–30), even if the latter is more relevant to the definition of a playing style. A starting value of k=4 was considered reasonable for the first run of the segmentation algorithm; the resulting groups were sufficiently distinct and sufficiently different in size that no further tuning of k was necessary to obtain an interpretable result.

Cluster	Share	Profile	Coaching Implication
Standard play	59%	Moderate pace, normal endings	Baseline group; general coaching applies
Clock-driven losses	29%	Fast time control, mostly time-forfeit endings	Needs clock-management training, not chess technique
Long grinders	10%	~123 moves average, 3x longer than other clusters	Would benefit from endgame technique coaching
Deliberate classical	2%	Long time controls, lowest average rating	Smaller, slower-paced segment

Cluster 1 - Standard Play (59% of games, 3.58 million).

This is the cluster of the majority of the games in this set, where the category is "dominant, default" - a moderate time control (about 5 minutes, but with a small increment), an average game length, and most important of all, every single game in this cluster is terminated with a "Normal" termination (checkmate, resignation or an agreed draw, not by the time limit). It is a cluster of chess played, played the way the game is traditionally understood, and which determines the contrast to the other three, more specific, clusters.

Cluster 2 – Clock Driven Losses (29% of games, 1.75 million).

This cluster is not so much determined by what is being played as the clock ticking. It has the shortest base time and smallest increment of any cluster, and the vast majority of games (almost all) do not result in a checkmate or a resignation, but in 'Time forfeit.' The business implication here is important precisely because it is counterintuitive: almost one-third of all the games in the dataset are lost not to a chess mistake, but to mismanagement of the clock. A player in this cluster doesn't necessarily understand the game any less than others, they simply have a weaker relationship with the clock, and no amount of opening theory or tactical training closes that particular gap.

Cluster 3 - Long Grinders (10% of games, 589,000)

There is one dominant number in here: the number of moves—about 123, compared with more than three times that of other clusters. They're played-out technical games, and a significant amount of time is probably spent in the endgame in which small positional advantages are realized over a lengthy period rather than quickly through tactic. Perhaps more importantly, although these games tend to begin with a decent amount of time on the clock, 28% of them still hit the time forfeit mark due to the length of the game not the fast time control as in Cluster 2. The kind of endgame technique coaching that might be most helpful for this cluster would be endgame technique coaching, as their games are likely to be in the stage of the game where endgame technique is most important.

Cluster 4 - Deliberate Classical (2% of games, 138,000).

The smallest cluster, distinguished by long base time controls (around 28 minutes) and a large per move increment (more like classical or serious rapid than blitz and bullet formats dominate the rest of this data) is closer to classical or serious rapid than the blitz and bullet formats. Almost never played to time in these games, which is a slower format. The slight difference that the lowest rating is in this cluster (1,675, compared to ~1,810-1,830 in the others) is one detail to not over-interpret; it's a sign that the cluster attracts a different type of player, possibly a more casual or newer segment of the player base, but not a sign of any kind of the format.

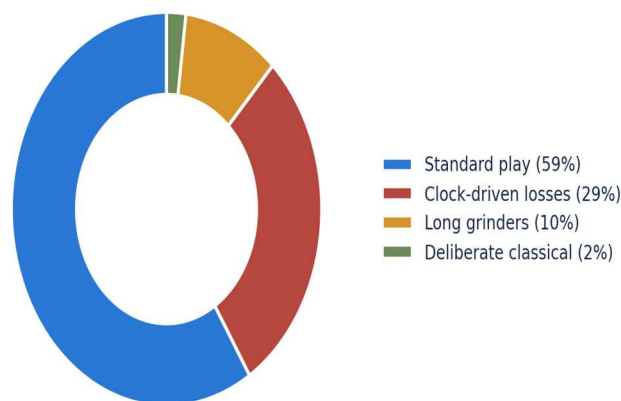


Figure 3- Relative size of the four playing-style clusters

EXTENDED ANALYSIS – THE CALIBRATION CURVE

All of Findings 1-3 is applicable to information you might find on a scoresheet - ratings, results, duration of games, and game ending. None of it does look at the game of chess play move by move. It can tell you that "stronger players convert advantages better" in general, it cannot answer the more specific question that this project actually began with: If a chess engine rates a position at a given value, what does that number mean to a player on a given rating level? To do so, a second, independent slice of the data was required — one that reflects not only the final score, but also the location of the player on the board.

METHODOLOGY

Out of the 6.1 million games, a smaller number, 765,433 (12.5%), have something extra attached: Stockfish's own opinion on the position after each player's move, when they played and then clicked on "analyze this game". Imagine it's like a director's commentary track on some DVDs of a movie.

For each of those games, the evaluation for each move was taken out separately — so for this game, you get a row for each move, which has two tags: the size of the advantage at that turn and the rating band of the better player. The advantages were then divided into seven simple buckets ranging from 'White is losing badly' to 'White is winning badly', including 'roughly even'. Then for each possible bucket and rating band, the question was, "how often, in all of the real games that ever played a situation like this, did White actually win?". There is no modeling magic here; it's just a huge number of moves and a counting of real events, and a grouping of events.

WHAT IT ACTUALLY SHOWS

When plotted the results take the form of an hourglass with the widest parts at the ends and the narrowest part in the middle. The centre (a roughly equal position) is neither good nor bad, as can be seen, every rating band is close to the same percentage (around 45-48%). This is sensible because there hasn't been a real advantage for anyone yet so skill level hasn't come into play yet. However, go off center and the bands will spread quickly. A 2400+ player can make it a real win 87.7% of the time in this winning position, but a player under 1200 is able to do the same only 76.3% of the time in this winning position. Flip it around: when a 2400+ player finds him/herself in a clearly losing position, only he/she recovers a victory 8.2% of the time, whereas an under-1200 player can somehow win it over in more than double that amount of time, 20.2% of the time.

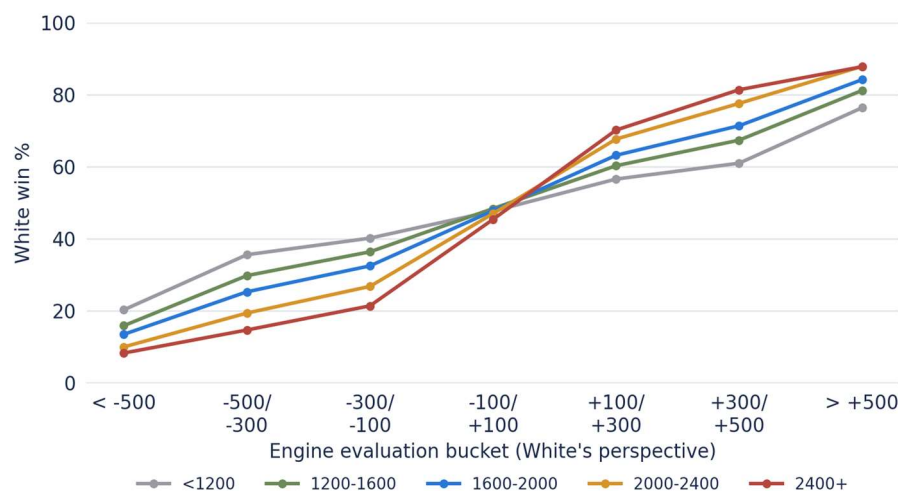


Figure 4 — Win probability by evaluation bucket and rating band, showing the "hourglass" convergence at balanced positions and divergence at decisive ones

Warm words: stronger players more adept at sealing a game, weaker players more inclined to squander a winning opportunity or to take an unearned opportunity when they are trailing. It's not enough to look at the number of the engine, it's about the historical outcome data that's overlaid on top of that.

WHY IT MATTERS

At the core of the whole project is the live demo that isn't merely a "the engine thinks you're winning" kind of answer, but a "people at your level, in this situation, actually win X% of the time. It's the whole product idea in one number, and it wouldn't exist if it weren't for this additional analysis.

As an example of how the table is applied to a live game in action, Figure 6 below depicts the live demonstration in mid-game when White is to move in the 1600-2000 rating range. In this case, the baseline win probability for the position is 48%, and all possible legal bishop moves are analyzed and sorted in real-time. The most prudent options — Bb3, and the set of central retreats (Bb5+, Bd3, Be2, Bf1) — are grouped around 48% and 32% respectively; this corresponds to the moves that maintain the current position in tact. Taking the piece with Bxd5 decreases to 25%, which accurately reflects the danger of gaining material while risking tempo or structural integrity of the position. The most dangerous move of all is Ba6, moving the bishop to a disconnected, undefended square, and is marked as "Blunder" at 13%, clearly showing it to be the worst option out there.



Figure 6 — The live demonstration ranking legal moves by rating-adjusted win probability, correctly identifying Bxd5 as risky and Ba6 as a blunder among otherwise safe alternatives.

And this one example alone demonstrates the main idea behind the project in practice: instead of just showing the static engine evaluation, the program presents a player at a certain rating range which of his or her available options are safe, which are dangerous and which are losing, this is a level of specificity a generic engine analysis does not provide.

BUSINESS INSIGHTS

- With one coaching platform offering the same guidance to all levels of skill, the underrepresentation of each end of the spectrum means that while the board advantage applies to both sides, it does not have the same value, which the coaching material must account for.
- Since almost a third of the games in this dataset were lost because of the way the clock was used and not because of actual chess skills, then a coaching platform that focuses on the opening theory and tactical plays would be tackling the wrong issue with a considerable part of its user base.
- The additional research of the win rate of certain openings across different rating bands revealed openings whose effectiveness varied based on the opponent's skill – with some doing much better against a lower rating and being punished when facing the more advanced opponents.

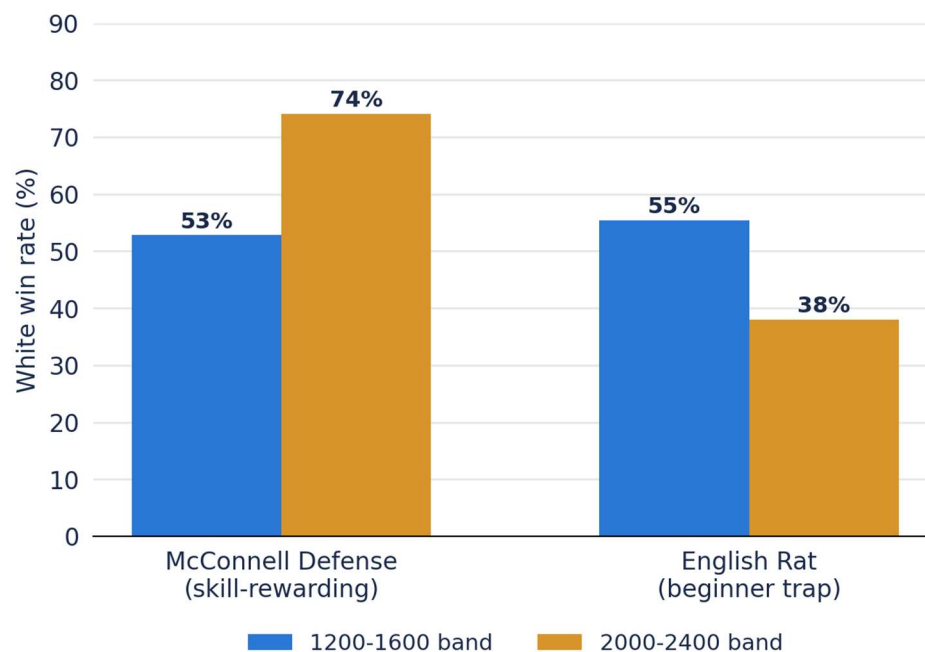


Figure 5 — Same opening, opposite outcome by rating band. The McConnell Defense performs better against stronger opposition, while the English Rat performs worse — evidence that some openings function differently depending on who they're played against.

STRATEGIC RECOMMENDATIONS

- Segment the coaching material and win probability feedback by rating band instead of providing one generic engine evaluation for everyone.
- Give more attention to clock management training as a separate product since it deals with the biggest failure cluster uncovered in the present analysis.
- Create an opening risk indicator to identify the lines where there is a high win probability difference by rating band so that players will know which of their openings may become ineffective once their skill level increases.
- Add moves to the evaluated-games pipeline to be able to provide popularity statistics on them in the next release (they are not in the current extraction).

RISKS & LIMITATIONS

- These results apply to a specific period of one month of activity on the site (June 2016); further verification needs to take place with up-to-date data before making recommendations for the live application.
- The rating band calibration is based on the 12.5% of games that have engine analysis done; this might not fully represent the overall population.
- The classification model accuracy (all three approaches are within half a point of each other) indicates how hard it is to make predictions in chess; this is something that should not be overstated.
- The move popularity and turning point by opening features, which can serve as valuable additions, have not been implemented because of time constraints and can be considered for future implementation.

APPENDIX

REFERENCES

Lichess Open Database — <https://database.lichess.org>

Lichess API Documentation — <https://lichess.org/api>

GitHub Repository — <https://github.com/akki-thegr8/chess-rating-analytics-GrpA01>

AI USAGE DISCLOSURE

For this project, Claude (Anthropic) was utilized as an assistant for coding and analysis throughout the stages of data engineering, modeling, and reporting. The code was verified by the team involved in the project, and all the conclusions made during analysis were compared to the actual results.

NAME	WORK DONE
Manav Karia	Data Acquisition & Preprocessing, sourced the Lichess archive, resolved download/decompression issues, built the initial parsing script that produced the clean 6.1M-game dataset
Arjun Bhatnagar	Data Engineering - Databricks/PySpark cleaning pipeline and core feature engineering (rating_diff, rating_band, outcome, one-hot encoding, time-control splitting)
Sarvesha Bhatgare	Machine Learning (Classification) - built and evaluated the naive baseline, logistic regression, and random forest models; investigated and explained the random forest underperformance
Aditi Chandra Yadav	Machine Learning (Clustering & Calibration) — built the KMeans clustering pipeline, interpreted the four archetypes, led the evaluated-games extraction and rating-band calibration table
Akshat Rathod	Live Demo Development, built the interactive Stockfish-powered chessboard and connected it to real Databricks-exported data
Jasreen Kour Bagga	Business Analysis & Documentation, translated findings into business recommendations, wrote the consulting report and slide deck, maintained the GitHub repository